

REAL-TIME CROWD-AWARE SMART ENVIRONMENTAL AUTOMATION SYSTEM

**A PROJECT REPORT
(PHASE – I)**

Submitted by

JANAGAN B	727723EUCS083
KUMARA VISHAL Y	727723EUCS110
MUKESH V	727723EUCS129
NARESH A N	727723EUCS139

*In partial fulfillment for the award of the degree
of*

BACHELOR OF ENGINEERING
in

COMPUTER SCIENCE AND ENGINEERING

SRI KRISHNA COLLEGE OF ENGINEERING AND TECHNOLOGY

**An Autonomous Institution | Approved by AICTE | Affiliated to Anna University | Accredited
by NAAC with A++ Grade, Kuniamuthur, Coimbatore – 641008.**

OCTOBER 2026



SRI KRISHNA COLLEGE OF ENGINEERING AND TECHNOLOGY

An Autonomous Institution | Anna University Affiliated | NAAC A++ Accredited

Kuniamuthur, Coimbatore – 641008

BONAFIDE CERTIFICATE

Certified that this project report **"REAL-TIME CROWD-AWARE SMART ENVIRONMENTAL AUTOMATION SYSTEM"** is the bonafide work of **"JANAGAN B (727723EUCS083), KUMARA VISHAL Y (727723EUCS110), MUKESH V (727723EUCS129), NARESH A N (727723EUCS139)"** who carried out the project work under my supervision.

SIGNATURE

Dr. N. KOUSIKA
HEAD OF THE DEPARTMENT
Associate Professor
Department of CSE
Sri Krishna College of Engineering
and Technology, Coimbatore – 641008.

SIGNATURE

Dr. N. KOUSIKA
SUPERVISOR
Associate Professor
Department of CSE
Sri Krishna College of Engineering
and Technology, Coimbatore – 641008.

Certified that the candidates were examined by us in the Project Phase I Viva Voce examination held on _____ at Sri Krishna College of Engineering & Technology, Kuniyamuthur, Coimbatore – 641008.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ABSTRACT

This project presents a Real-Time Crowd-Aware Smart Environmental Automation System that integrates computer vision, and machine learning for intelligent crowd monitoring and environmental control. YOLOv8 is used for person detection, DeepSORT for multi-object tracking, and a ResNet-based model for crowd density estimation using the ShanghaiTech dataset. The system dynamically controls devices such as fans, lights, and displays based on crowd conditions. A Flask backend manages the system APIs, while a React.js frontend provides a real-time dashboard for monitoring crowd density, environmental conditions, device status, and alerts. The system follows a real-time pipeline in which detected crowd information and environmental data are processed by a decision engine to determine appropriate control actions. This enables the system to respond dynamically to changes in occupancy and environmental conditions instead of relying on fixed control settings. The proposed system achieves 91.2% crowd-counting accuracy, 94.2% mAP for crowd detection, and 45 FPS processing performance. The project demonstrates the potential of combining AI-based crowd analysis, Flask-based backend services, and React.js visualization to create scalable and energy-efficient smart environments for commercial buildings, educational institutions, and public venues. Index Terms— Crowd Analysis, YOLOv8, DeepSORT, ResNet, IoT, Raspberry Pi, DHT11, React.js, Flask, Smart Environmental Automation.

ACKNOWLEDGEMENT

First and foremost, we thank the Almighty for being our light and for showering His gracious blessings throughout the course of this project. We express our gratitude to our beloved Principal, **Dr. K. Porkumaran**, for providing us with all the necessary facilities and support.

With a grateful heart, we extend our sincere thanks to our Head of the Department, **Dr. N. Kousika**, Department of Computer Science and Engineering, for her motivation and support to complete the project work.

We extend our sincere thanks to our project supervisor, **Dr. N. Kousika**, Associate Professor, Department of Computer Science and Engineering, for her valuable guidance, constructive suggestions, and continuous motivation throughout the development of this project, which helped us improve our skills.

We thank **Ms. Jayapradha A**, Assistant Professor, Department of Computer Science and Engineering and Project Coordinator, for her motivation, coordination, and support.

We are thankful to all the **Teaching and Non-Teaching Staff** of the Department of Computer Science and Engineering and to all those who have directly and indirectly extended their help to us in completing this project work successfully.

We extend our sincere thanks to our **family members, friends, and teammates**, who have been strongly supporting us in all our endeavours and encouraged us throughout the successful completion of this project.

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	COVER PAGE	i
	BONAFIDE CERTIFICATE	ii
	ABSTRACT	iii
	ACKNOWLEDGEMENT	iv
	TABLE OF CONTENTS	v
	LIST OF FIGURES	x
	LIST OF TABLES	xi
	LIST OF ABBREVIATIONS	xii
1	INTRODUCTION	1
	1.1 General / Overview	1
	1.2 Problem Statement	1
	1.3 Objectives	2
	1.4 Scope of the Project	3
	1.5 Contributions	3
	1.6 Summary	4
2	LITERATURE REVIEW	5
	2.1 General	5
	2.2 Computer Vision for Crowd Analysis	5

	2.3 IoT-Enabled Environmental Sensing Networks	6
	2.4 Adaptive Control Strategies and Optimization	7
	2.5 Integrated System Architectures	7
	2.6 Comparative Analysis	8
	2.7 Research Gap	8
	2.8 Summary	9
3	SYSTEM ANALYSIS	10
	3.1 General	10
	3.2 Existing System	10
	3.3 Proposed System	11
	3.4 Feasibility Study	12
	3.5 Functional Requirements	12
	3.6 Non-Functional Requirements	14
	3.7 Hardware Requirements	15
	3.8 Software Requirements	15
	3.9 Summary	16
4	SYSTEM DESIGN	17
	4.1 System Architecture	17
	4.2 Crowd Detection Design	18
	4.3 Crowd Density Estimation Design	19

	4.4 Person Tracking and Re-Identification	21
	4.5 Environmental Sensing Design	22
	4.6 Decision Engine Design	22
	4.7 React.js Frontend Design	23
	4.8 Flask Backend and API Design	24
	4.9 Data Logging Design	24
	4.10 UML Diagrams	25
	4.11 Summary	27
5	IMPLEMENTATION	28
	5.1 System Specification	28
	5.2 Dataset Preparation and Preprocessing	29
	5.3 Crowd Detection Implementation	30
	5.4 Crowd Density Estimation Implementation	30
	5.5 DeepSORT Tracking Implementation	31
	5.6 IoT and Sensor Implementation	32
	5.7 Decision Engine Implementation	33
	5.8 P React.js Frontend Implementation	33
	5.9 Flask Backend Implementation	34

	5.10 Summary	35
6	RESULTS AND DISCUSSION	36
	6.1 System Results	36
	6.2 Crowd Detection Results	36
	6.3 Crowd Density Estimation Results	37
	6.4 Person Tracking Results	39
	6.5 Environmental Monitoring Results	39
	6.6 Environmental Control Results	40
	6.7 React.js Dashboard Output	40
	6.8 Performance Analysis	41
	6.9 Comparative Analysis	42
	6.10 Discussion	43
7	CONCLUSION AND FUTURE ENHANCEMENTS	44
	7.1 Conclusion	44
	7.2 Limitations	45
	7.3 Future Enhancements	45

SDG MAPPING	46
REFERENCES	48
APPENDICES	52
APPENDIX I	52
APPENDIX II	58
APPENDIX III	59
APPENDIX IV	60

LIST OF FIGURES

Figure 3.1 System Flow	12
Figure 4.1 System Architecture	18
Figure 4.2 Proposed Custom Architecture of the DensityResNet	21
Figure 4.3 UML Use Case Diagram	25
Figure 4.4 UML Sequence Diagram	26
Figure 4.5 UML Activity Diagram	26
Figure 6.1 Density Estimation Performance Comparison	38
Figure 6.2 CCTV Captured Crowded Image	38
Figure 6.3 Calculated Density Using Proposed System	38
Figure 6.4 React.js Dashboard Output	41
Figure 6.5 Dashboard Output	60
Figure 6.6 Home Controls Output	60
Figure 6.7 Crowd Detection Output	61
Figure 6.8 Admin Login Output	61
Figure 6.9 Working Models Output	62
Figure 6.10 Analytics & logs Output	62

LIST OF TABLES

Table 2.1 Comparative Analysis of Existing Systems	8
Table 3.1 Hardware Requirements	15
Table 3.2 Software Requirements	15
Table 4.1 Training Configuration For Shanghaitech Dataset	19
Table 4.2 Enhanced Densityresnet Architecture	20
Table 4.3 Re-Id Network Configuration	22
Table 4.4 Iot Device Specifications	22
Table 4.5 Context Factors	23
Table 4.6 Time Factors	23
Table 4.7 API Endpoints	24
Table 4.8 Logging Structure	25
Table 5.1 System Implementation Specification	28
Table 5.2 Preprocessing Parameters	29
Table 5.3 Main API structure	34
Table 6.1 Occlusion Handling Evaluation	37
Table 6.2 Real-Time Processing Pipeline Performance	41
Table 6.3 Comparative Performance Analysis	42

LIST OF ABBREVIATIONS

ABBREVIATION	EXPANSION
AI	Artificial Intelligence
API	Application Programming Interface
ASPP	Atrous Spatial Pyramid Pooling
CBAM	Convolutional Block Attention Module
CNN	Convolutional Neural Network
CPU	Central Processing Unit
CSV	Comma-Separated Values
CSP	Cross Stage Partial
CV	Computer Vision
DHT	Digital Humidity and Temperature
DNN	Deep Neural Network
FPS	Frames Per Second
FPN	Feature Pyramid Network
GPIO	General-Purpose Input/Output
GPU	Graphics Processing Unit
IoT	Internet of Things
IoU	Intersection over Union
JSON	JavaScript Object Notation
L2	L2 Normalization
LSTM	Long Short-Term Memory
MAE	Mean Absolute Error
MAPE	Mean Absolute Percentage Error

mAP	Mean Average Precision
mAP@0.5	Mean Average Precision at IoU Threshold 0.5
ML	Machine Learning
MPS	Metal Performance Shaders
NMS	Non-Maximum Suppression
PAN	Path Aggregation Network
PSNR	Peak Signal-to-Noise Ratio
RAM	Random Access Memory
Re-ID	Re-Identification
REST	Representational State Transfer
RH	Relative Humidity
RGB	Red, Green, Blue
SDG	Sustainable Development Goal
SSIM	Structural Similarity Index Measure
UML	Unified Modeling Language
YOLO	You Only Look Once

CHAPTER 1

INTRODUCTION

1.1 GENERAL / OVERVIEW

The increasing number of people in public and indoor spaces creates challenges in maintaining suitable environmental conditions and efficient energy usage. Conventional environmental control systems generally depend on fixed schedules or basic occupancy sensors, which cannot effectively respond to rapid changes in crowd density. This can result in unnecessary energy consumption while also affecting occupant comfort.

The proposed Real-Time Crowd-Aware Smart Environmental Automation System combines Internet of Things (IoT), Computer Vision, and Machine Learning to provide intelligent and adaptive environmental control. The system uses YOLOv8 for person detection, DeepSORT for multi-object tracking, and a ResNet-based crowd density estimation model trained using the ShanghaiTech Crowd Counting Dataset. Environmental parameters such as temperature and humidity are monitored using DHT11 sensors connected to a Raspberry Pi 4. Based on crowd density and environmental conditions, the system can control devices such as fans, lighting systems, and display units. The system also provides a web-based monitoring platform with a Flask backend and React.js frontend, allowing users to view real-time crowd information, environmental conditions, device status, and system alerts

1.2 PROBLEM STATEMENT

Traditional environmental control systems mainly rely on fixed schedules or simple occupancy sensors. These systems are unable to accurately understand real-time crowd variations and therefore cannot dynamically adjust environmental conditions according to the number of people present. This may lead to overcooling

or insufficient ventilation in crowded areas, resulting in energy wastage and reduced occupant comfort. Another challenge is the integration of crowd analysis with environmental sensing and automated control. Existing approaches often treat crowd monitoring, environmental sensing, and device control as separate components. The proposed project addresses this problem by integrating computer vision, machine learning, IoT sensors, and an intelligent decision-making mechanism into a unified system. The system continuously analyzes crowd conditions and environmental parameters to support adaptive control of connected devices.

1.3 OBJECTIVES

The overarching goal of the Real-Time Crowd-Aware Smart Environmental Automation System is to develop an intelligent ecosystem that monitors crowd density and environmental conditions. The specific objectives include:

- To develop a real-time crowd detection system using YOLOv8.
- To track individuals across video frames using DeepSORT.
- To estimate crowd density using a ResNet-based density estimation model trained on the ShanghaiTech dataset.
- To monitor environmental parameters such as temperature and humidity using DHT11 sensors.
- To integrate the sensing and computer vision modules with a Raspberry Pi 4 edge computing platform.
- To develop an intelligent decision mechanism that uses crowd and environmental information for adaptive control.
- To automatically control devices such as fans, lights, and display systems according to system conditions.
- To develop a Flask backend and React.js frontend dashboard for real-time monitoring and control.

- To reduce unnecessary energy consumption while maintaining suitable environmental conditions.

These objectives correspond to the report's proposed integration of crowd detection, density estimation, IoT sensing, intelligent decision-making, and adaptive environmental control

1.4 SCOPE OF THE PROJECT

The scope of the project covers the development of an integrated AI and IoT-based crowd-aware environmental automation system. The system can process camera feeds to detect and track people, estimate crowd density, and combine this information with environmental measurements such as temperature and humidity. The project includes the use of YOLOv8, DeepSORT, ResNet-based density estimation, DHT11 sensors, Raspberry Pi, Flask APIs, and React.js. The React.js dashboard provides visualization of crowd density, environmental parameters, device status, and alerts, while the Flask backend manages communication between the different system components. The system can be applied to environments such as commercial buildings, educational institutions, retail spaces, and public venues, where crowd levels and environmental conditions can change dynamically. The project primarily focuses on real-time monitoring, analysis, and adaptive control rather than replacing complete building-management systems.

1.5 CONTRIBUTIONS

This project makes several significant contributions to smart environmental automation by enabling real-time crowd monitoring:

- Integration of YOLOv8 and DeepSORT for real-time person detection and tracking

- Development and application of a Multi-Scale DensityResNet approach for crowd density estimation.
- Development of an intelligent decision mechanism that combines crowd density and environmental conditions for adaptive device control.
- Implementation of a Flask-based backend with REST APIs for communication between system modules.
- Development of a React.js-based frontend dashboard for real-time visualization and control.
- Integration of crowd analytics with environmental devices such as fans, lighting, and display systems.
- Evaluation of the system using crowd detection, density estimation, tracking, processing-speed, and environmental-control metrics.

The report presents results including 91.2% crowd-counting accuracy, 94.2% mAP for the optimized YOLOv8-Crowd system, and 45 FPS processing performance.

1.6 SUMMARY

This chapter introduced the Real-Time Crowd-Aware Smart Environmental Automation System and discussed the need for intelligent environmental control based on real-time crowd conditions. The limitations of conventional fixed-control systems were identified, particularly their inability to adapt to changing occupancy levels.

The proposed system addresses these limitations by integrating YOLOv8, DeepSORT, ResNet-based crowd density estimation, IoT sensors, Raspberry Pi, Flask backend, and React.js frontend into a unified platform. The system aims to provide real-time crowd monitoring, environmental sensing, intelligent decision-making, and automated device control

CHAPTER 2

LITERATURE REVIEW

2.1 GENERAL

Crowd management and environmental automation have become important areas of research with the increasing use of smart buildings, public spaces, educational institutions, commercial facilities, and other crowded environments. Traditional environmental control systems generally depend on fixed schedules or basic occupancy sensors, which may not respond effectively to continuous changes in crowd density. This can lead to unnecessary energy consumption and reduced environmental comfort.

2.2 COMPUTER VISION FOR CROWD ANALYSIS

Computer vision has become an important approach for automated crowd monitoring and analysis. The YOLO family of object detection models has been widely used for real-time person detection. Patankar et al. [2] implemented YOLOv3 on ESP32 microcontrollers and reported a detection accuracy of 96.31% with an inference delay of 1.2 seconds. However, its performance decreased to 78.2% in high-density scenes with more than three persons per square meter due to occlusion. Kumar et al. [3] integrated YOLOv8 with DeepSORT on Raspberry Pi 4 and achieved 91.7% mAP at 18 FPS, while reducing identity switches by 34% compared with YOLOv8-only approaches.

Crowd density estimation techniques have also been extensively studied for highly populated environments where individual detection becomes difficult. CSRNet [4] demonstrated the effectiveness of density-based approaches on the ShanghaiTech dataset, while improvements using dilated convolutions reduced computational complexity. Hybrid approaches such as the YOLOv8-CSRNet

pipeline proposed by Li et al. [9] combined object detection and density estimation and achieved 89.2% accuracy on embedded hardware. However, performance still decreased in extremely dense scenes. To support deployment on resource-constrained devices, researchers have also explored quantization, knowledge distillation, TensorFlow Lite, and OpenVINO optimization techniques. These approaches demonstrate the importance of balancing detection accuracy, computational requirements, and real-time performance for edge-based crowd analysis.

2.3 IOT-ENABLED ENVIRONMENTAL SENSING NETWORKS

IoT-based environmental sensing networks provide continuous monitoring of environmental parameters required for intelligent automation. DHT11 and DHT22 sensors are commonly used for temperature and humidity measurement because of their low cost, while advanced sensors such as BME680 provide improved accuracy and additional environmental measurements. Research has also shown that appropriate sensor placement can improve spatial monitoring of environmental conditions. Low-cost combinations of occupancy and environmental sensors, such as PIR and DHT11, have been investigated for smart monitoring applications.

Communication technologies also play an important role in IoT-based systems. MQTT over Wi-Fi is commonly used for communication because of its low latency and suitability for indoor environments. Edge computing can further reduce the delay between sensing and actuation by processing information closer to the devices rather than depending entirely on cloud infrastructure. However, sensor reliability remains a challenge because sensors such as DHT11 may experience measurement drift over time and require periodic recalibration. These studies demonstrate the importance of reliable sensing, communication, and edge processing in real-time environmental automation

2.4 ADAPTIVE CONTROL STRATEGIES AND OPTIMIZATION

Adaptive control systems are designed to adjust environmental conditions according to changes in occupancy and surrounding conditions. Traditional binary occupancy-based control systems may cause energy wastage during partial occupancy, while temperature-only control approaches may fail to maintain suitable comfort levels. Multi-threshold rule-based control systems provide improved adaptability by considering multiple environmental parameters and have demonstrated reductions in energy consumption.

Machine learning-based approaches have also been explored for intelligent environmental control. Reinforcement learning methods such as Q-learning have demonstrated energy savings in HVAC systems, while multi-objective models consider energy consumption, comfort, and safety simultaneously. Predictive models such as LSTM have also been investigated to improve environmental comfort. Hybrid architectures that combine rule-based fallback mechanisms with machine learning optimization provide greater robustness and adaptability. These approaches demonstrate the potential of intelligent control systems to balance energy efficiency with environmental comfort.

2.5 INTEGRATED SYSTEM ARCHITECTURES

Integrating computer vision, IoT sensing, machine learning, and environmental control provides a comprehensive approach to smart environmental management. However, such integration introduces challenges related to computational requirements, communication latency, synchronization, and hardware compatibility. Crowd-analysis models can require significant processing resources, while IoT sensors continuously generate environmental data that must be processed together with visual information. Edge computing provides an effective approach for reducing processing and communication latency by performing

analysis closer to the data source. An integrated architecture can therefore combine real-time crowd detection and tracking with environmental sensing to generate appropriate control decisions. Such architectures can support applications in smart buildings, commercial spaces, educational institutions, and public venues where crowd levels and environmental conditions change dynamically.

2.6 COMPARATIVE ANALYSIS

To validate the necessity and effectiveness of the proposed system, a comparative analysis was conducted against existing environmental monitoring solutions and conventional automation approaches in Table 2.1.

TABLE 2.1: COMPARATIVE ANALYSIS OF EXISTING SYSTEMS

Approach	Detection	Tracking	Density	IoT	Real-Time
Traditional system	Basic	✗	✗	✓	Limited
YOLO-based	✓	Limited	✗	✗	✓
YOLO + DeepSORT	✓	✓	Limited	✗	✓
Density estimation	Limited	✗	✓	✗	Limited
Proposed system	✓	✓	✓	✓	✓

2.7 RESEARCH GAP

The literature identifies several gaps in existing crowd-aware environmental automation systems. A large proportion of crowd-analysis models rely on GPU-based processing, while relatively fewer models are optimized for ARM-based edge

devices. Simplifying models for edge deployment can also result in accuracy losses. Furthermore, many existing approaches are evaluated through simulations rather than real-world deployments, and a significant portion of studies use outdoor datasets that may not fully represent indoor crowd environments. The literature also reports limited evaluation of the relationship between energy consumption and environmental comfort. Integration challenges such as sensor-vision calibration, synchronization errors, and long-term system reliability remain important areas requiring further investigation.

2.8 SUMMARY

This chapter reviewed existing research related to computer vision-based crowd analysis, IoT-enabled environmental sensing, adaptive control, and integrated smart-system architectures. Existing studies demonstrate the effectiveness of YOLO-based detection, DeepSORT tracking, density estimation models, IoT sensors, and adaptive control techniques. However, challenges remain in handling dense crowds, optimizing models for edge devices, integrating multiple data sources, and achieving reliable real-time environmental control. These gaps motivate the proposed Real-Time Crowd-Aware Smart Environmental Automation System, which integrates YOLOv8, DeepSORT, ResNet-based crowd density estimation, IoT sensing, Raspberry Pi, Flask backend, and React.js frontend into a unified platform.

CHAPTER 3

SYSTEM ANALYSIS

3.1 GENERAL

System analysis is an important phase in the development of the Real-Time Crowd-Aware Smart Environmental Automation System. It involves studying the limitations of existing environmental automation approaches and defining the requirements of the proposed system. The proposed system integrates computer vision, machine learning, IoT-based environmental sensing, edge computing, and web-based monitoring to provide adaptive environmental control based on real-time crowd conditions. The system uses YOLOv8 for person detection, DeepSORT for tracking, a ResNet-based model for crowd density estimation, and DHT11 sensors for monitoring temperature and humidity. The processed information is used by the decision engine to generate suitable control actions for environmental devices. A Flask backend manages the application logic and APIs, while the React.js frontend provides a user-friendly interface for monitoring and controlling the system.

3.2 EXISTING SYSTEM

Traditional environmental automation systems generally depend on fixed schedules, predefined temperature settings, or basic occupancy sensors. Such systems provide limited awareness of the actual number of people present in an environment and cannot accurately respond to rapid changes in crowd density. Basic occupancy sensors can determine whether a space is occupied, but they generally do not provide detailed information about the number of people or their distribution.

Existing computer-vision-based approaches can perform person detection, tracking, or crowd-density estimation, but these functions are often implemented independently rather than being integrated with environmental control. Similarly,

IoT-based systems can monitor environmental parameters such as temperature and humidity, but they may not consider real-time crowd conditions while making control decisions. These limitations can result in unnecessary energy consumption, delayed environmental response, and inconsistent occupant comfort. The project report identifies the lack of integrated crowd awareness, environmental sensing, and adaptive control as an important limitation of conventional approaches.

3.3 PROPOSED SYSTEM

The proposed system provides an integrated approach for real-time crowd-aware environmental automation. A camera captures the environment and the computer vision pipeline detects people using YOLOv8. DeepSORT maintains the identities of detected individuals across consecutive frames, helping to avoid repeated counting. A ResNet-based density estimation model provides additional information about crowd density, particularly in situations involving crowded or partially occluded regions. At the same time, DHT11 sensors collect environmental parameters such as temperature and humidity. The crowd information and environmental sensor readings are processed by the intelligent decision engine. Based on these inputs, the system generates adaptive control actions for devices such as fans, lighting systems, and display units. The Raspberry Pi acts as an edge computing platform for processing and control. The Flask backend provides REST APIs for communication between the system components, while the React.js frontend displays crowd information, environmental readings, system status, and control information through a web dashboard.

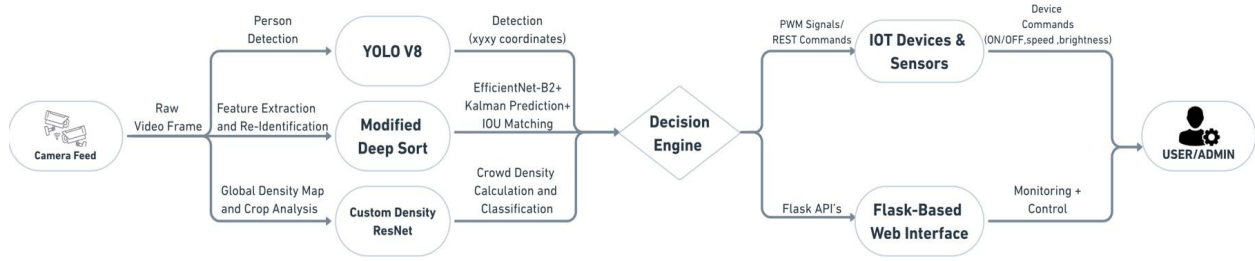


Figure 3.1: System Flow

3.4 FEASIBILITY STUDY

The feasibility study determines whether the proposed system can be practically developed and implemented using the available technologies and resources:

- **Technical Feasibility:** The system is technically feasible because it uses established technologies such as YOLOv8, DeepSORT, ResNet-based deep learning, Python, Flask, React.js, and Raspberry Pi. The project also uses DHT11 sensors for environmental monitoring. The presentation describes the integration of these components into an end-to-end system for real-time monitoring and automated control.
- **Economic Feasibility:** The system can be developed using relatively affordable hardware components such as Raspberry Pi, DHT11 sensors, cameras, and basic environmental control devices. The use of open-source software frameworks and machine-learning libraries also reduces software development costs.
- **Operational Feasibility:** The proposed system is designed to operate with minimum manual intervention. Environmental data and crowd information are processed automatically, and the decision engine generates appropriate control actions. The React.js dashboard allows users to monitor system conditions and view relevant information through a centralized interface.
- **Schedule Feasibility:** The system can be developed incrementally by implementing the computer vision models, sensor integration, backend, frontend,

decision engine, and testing as separate development stages. This modular approach supports systematic development and integration.

3.5 FUNCTIONAL REQUIREMENTS

The functional requirements define the operations that the proposed system must perform.

- The system shall capture real-time video from a camera.
- The system shall detect people using the YOLOv8 model.
- The system shall track detected individuals across video frames using DeepSORT.
- The system shall estimate crowd density using the ResNet-based density estimation model.
- The system shall collect temperature and humidity information using DHT11 sensors.
- The system shall process crowd and environmental information through the decision engine.
- The system shall generate adaptive control actions based on crowd and environmental conditions.
- The system shall communicate control and monitoring information through the Flask backend.
- The system shall display real-time system information through the React.js dashboard.
- The system shall maintain logs containing relevant monitoring information such as timestamps, crowd counts, density values, sensor readings, and device status.

- The system shall allow users to monitor the current environmental and crowd conditions through the dashboard.

3.6 NON-FUNCTIONAL REQUIREMENTS

The non-functional requirements define the performance, usability, reliability, and other quality characteristics expected from the system.

Performance: The system should process crowd information with low latency to support real-time monitoring and control.

Scalability: The architecture should allow additional sensors, environmental devices, and monitoring features to be incorporated without requiring major changes to the complete system.

Reliability: The system should provide consistent detection, tracking, environmental monitoring, and control operations during continuous execution.

Usability: The React.js dashboard should provide a clear and user-friendly interface for viewing crowd conditions, sensor readings, and system status.

Maintainability: The system should use modular components so that individual components such as the detection model, backend APIs, frontend, or decision engine can be modified independently.

Security: Communication between the frontend and backend should be handled through controlled API endpoints, and system access should be appropriately managed.

Real-Time Responsiveness: Changes in crowd density and environmental conditions should be processed quickly enough to allow the system to generate timely control responses.

3.7 HARDWARE REQUIREMENTS

The major hardware components required for the proposed system are. Table 3.1 and Table 3.2 outline the software and hardware specifications respectively.

TABLE 3.1: HARDWARE REQUIREMENTS

Hardware Component	Purpose
Raspberry Pi 4	Edge computing and system control
Camera	Real-time video capture for crowd analysis
DHT11 Sensor	Temperature and humidity monitoring
Environmental Control Devices	Fan, lighting and other control outputs
Microcontroller/Control Interface	Interfacing control signals with environmental devices
Power Supply	Provides power to the Raspberry Pi, sensors and connected devices
Network Connectivity	Communication between system components and dashboard

3.8 SOFTWARE REQUIREMENTS

The software requirements for implementing the system include:

TABLE 3.2: SOFTWARE REQUIREMENTS (DEVELOPMENT)

Software / Technology	Purpose
Python	Backend, machine learning and system logic
Flask	REST API and backend services

React.js	Web-based frontend dashboard
YOLOv8	Real-time person detection
DeepSORT	Multi-object tracking and identity preservation
ResNet-based Model	Crowd density estimation
PyTorch / Deep Learning Libraries	Model development and execution
OpenCV	Image and video processing
HTML/CSS/JavaScript	Supporting web technologies for the React-based interface
CSV	Lightweight logging of system measurements and events
Development Environment	Model training, testing and application development

3.9 SUMMARY

This chapter analyzed the requirements and feasibility of the Real-Time Crowd-Aware Smart Environmental Automation System. The limitations of traditional fixed-control and basic occupancy-based systems were identified, particularly their inability to respond effectively to changing crowd conditions. The proposed system addresses these limitations by combining YOLOv8-based detection, DeepSORT tracking, ResNet-based density estimation, DHT11 environmental sensing, Raspberry Pi edge processing, and an intelligent decision engine. The functional and non-functional requirements were defined along with the required hardware and software components. These requirements provide the foundation for the detailed system architecture and design presented in the next chapter.

CHAPTER 4

SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

The proposed Real-Time Crowd-Aware Smart Environmental Automation System follows a hybrid architecture that integrates computer vision, machine learning, IoT-based environmental sensing, edge computing, intelligent decision-making, and web-based monitoring. The architecture consists of four primary modules: the Enhanced Computer Vision Pipeline, Environmental Sensing Network, Intelligent Decision Engine, and Adaptive Environmental Control System. These modules work together to analyze crowd conditions, monitor environmental parameters, generate control decisions, and operate connected environmental devices.

The system operates on a Raspberry Pi 4 edge computing platform. The camera provides real-time video input for crowd analysis, while DHT11 sensors provide temperature and humidity measurements. The computer vision pipeline performs person detection using YOLOv8, identity tracking using DeepSORT, and crowd density estimation using the Multi-Scale DensityResNet model. The resulting crowd information is combined with environmental sensor readings by the decision engine. Based on contextual and temporal factors, the system generates adaptive control commands for environmental devices such as fans and lighting systems.

The web-based application provides a centralized interface for monitoring and interaction. In the implemented system, the React.js frontend communicates with the Flask backend, which manages APIs, processing logic, sensor information, device control, and system data. The architecture therefore provides an end-to-end flow from sensing and analysis to intelligent environmental control.

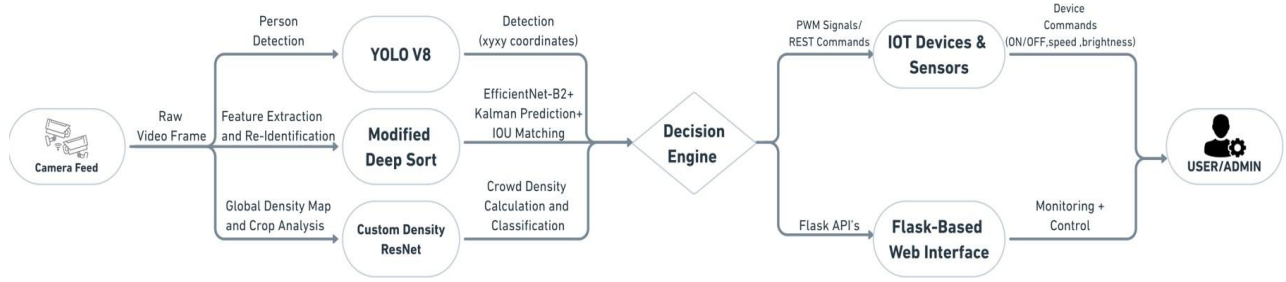


Figure 4.1 System Architecture of Crowd-Aware Smart Environmental Automation System

4.2 CROWD DETECTION DESIGN

The crowd detection component is designed using an enhanced YOLOv8 architecture optimized for crowded environments. The detection network incorporates modifications intended to improve feature extraction and detection performance under conditions involving occlusion, scale variation, and dense person distribution.

The architecture uses an optimized Focus Layer as an initial feature extraction component, followed by enhanced CSP modules. Attention mechanisms are incorporated into the early-stage CSP blocks, while deeper stages employ Squeeze-and-Excitation and CBAM-based attention mechanisms. An SPP-ELAN module is used to capture multi-scale contextual information. The feature fusion stage uses a hybrid PAN/FPN structure to facilitate bidirectional feature propagation and preserve fine-grained information.

TABLE 4.1: TRAINING CONFIGURATION FOR SHANGHAITECH DATASET

Parameter	Value	Description
Input Resolution	640×640	Optimal for YOLOv8
Batch Size	16	With gradient accumulation
Learning Rate	0.01	Cosine annealing
Weight Decay	0.0005	L2 regularization
Momentum	0.937	SGD momentum
IoU Threshold	0.7	Detection threshold
Confidence Threshold	0.25	Minimum confidence
Warmup Epochs	3	Learning rate warmup
Total Epochs	100	Complete training cycles

4.3 CROWD DENSITY ESTIMATION DESIGN

The crowd density estimation component uses a Multi-Scale DensityResNet architecture. The model extends the ResNet architecture with multi-scale feature extraction, Atrous Spatial Pyramid Pooling (ASPP), CBAM attention, and a progressive upsampling decoder. These components enable the network to capture contextual information at different spatial scales and reconstruct detailed density maps.

The density map is generated from annotated head positions using adaptive Gaussian convolution. The standard deviation of the Gaussian kernel is adjusted

according to perspective information so that density estimation can account for differences in apparent person size across the image.

The ground-truth density map is represented as:

$$\rho(x,y)=\sum_{i=1}^N\delta(x-x_i,y-y_i) * G_{\sigma_i}(x,y)$$

where the Gaussian standard deviation is determined using:

$$\sigma_i=\max(\sigma_{\min},\alpha d_i+\beta)$$

The report specifies $\alpha = 0.3$, $\beta = 2.0$, and $\sigma_{\min} = 1.5$.

TABLE 4.2: ENHANCED DENSITYRESNET ARCHITECTURE

Module	Output Size	Channels	Parameters	Enhancements	Module
Stem Conv	320×320×64	64	9.4K	7×7 Conv + BN	Stem Conv
ResBlock 1	320×320×256	256	215K	Bottleneck	ResBlock 1
ResBlock 2	160×160×512	512	1.22M	Stride 2	ResBlock 2
ResBlock 3	80×80×1024	1024	7.1M	Dilated Conv	ResBlock 3
ResBlock 4	80×80×2048	2048	14.96M	Atrous Conv	ResBlock 4
ASPP	80×80×256	256	2.5M	Multi-rate	ASPP
Attention	80×80×256	256	131K	CBAM Module	Attention
Decoder	320×320×1	1	850K	Progressive Up	Decoder



Figure 4.2 Proposed Custom Architecture of the DensityResNet

4.4 PERSON TRACKING AND RE-IDENTIFICATION

The person tracking module uses an enhanced DeepSORT framework to maintain individual identities across consecutive frames. The tracking system combines motion information from the Kalman filter with appearance information obtained from a re-identification network.

The appearance feature extractor uses EfficientNet-B2 and generates 256-dimensional L2-normalized feature vectors. The system uses triplet loss with hard negative mining to learn discriminative appearance representations. Multi-dataset training using Market-1501 and DukeMTMC is specified in the report for improving appearance generalization.

The tracking mechanism further uses adaptive Kalman filter parameters based on crowd density and occlusion conditions. The association cost combines IoU, appearance, and motion information using weighted coefficients.

TABLE 4.3: RE-ID NETWORK CONFIGURATION

Component	Specification	Value
Backbone	EfficientNet-B2	Pre-trained
Feature Dimension	256-D	L2 normalized
Triplet Loss Margin	0.3	Hard mining
Center Loss Weight	0.003	Feature clustering
Training Datasets	Market-1501, DukeMTMC	Multi-domain

4.5 ENVIRONMENTAL SENSING DESIGN

The environmental sensing subsystem monitors temperature and humidity using DHT11 sensors. The sensors are interfaced with the Raspberry Pi through GPIO connections and can be distributed throughout the monitored environment to provide environmental measurements.

TABLE 4.4: IOT DEVICE SPECIFICATIONS

Parameter	DHT11 Specification	Measurement Range
Temperature	$\pm 2^{\circ}\text{C}$ accuracy	0°C to 50°C
Humidity	$\pm 5\%$ RH accuracy	20% to 90% RH
Sampling Rate	1 Hz maximum	Every 2 seconds
Interface	Single-wire digital	GPIO compatible
Power Consumption	2.5 mA average	3.3V–5V supply

4.6 DECISION ENGINE DESIGN

The decision engine combines crowd density, environmental measurements, location context, and time-related information to generate adaptive environmental control actions.

The dynamic threshold is represented as:

$$\tau_{\text{dynamic}} = \tau_{\text{base}} \times \text{context_factor} \times \text{time_factor}$$

TABLE 4.5: CONTEXT FACTORS

Environment	Context Factor
Outdoor area	1.2
Indoor space	1.0
Transit zone	0.8

TABLE 4.6: TIME FACTORS

Time Condition	Time Factor
Peak hours (9–12, 14–17)	1.3
Normal hours	1.0
Off-peak hours	0.7

Control equations

Fan speed:

$$S_{\text{fan}} = \min(100, \max(20, 150 \times p_{\text{global}} + 30 \times p_{\text{trend}}))$$

Light intensity:

$$I_{\text{light}} = \min(100, \max(40, 200 \times p_{\text{global}} + 20 \times \text{visibility}))$$

Temperature control:

$$T_{\text{set}} = T_{\text{base}} - 2 \times \log(1 + 10 \times p_{\text{global}})$$

4.7 REACT.JS FRONTEND DESIGN

The frontend is implemented using React.js and provides the user-facing monitoring and control interface. The dashboard is designed to display real-time crowd information, environmental measurements, device status, and alerts.

The main dashboard components include:

1. Live video visualization with detection overlays.
2. Crowd count and density information.
3. Temperature and humidity monitoring.
4. Environmental charts.
5. Device status indicators.
6. Manual device-control options.
7. Configurable alerts and notifications.

4.8 FLASK BACKEND AND API DESIGN

The backend is implemented using the Flask framework and provides RESTful APIs connecting the React.js frontend with the computer vision, sensor, decision, and control components.

TABLE 4.7: API ENDPOINTS

API Endpoint	Purpose
/api/crowd/density	Provides real-time crowd analytics
/api/environment/sensors	Provides environmental sensor data
/api/control/devices	Manages device status and commands
/api/analytics/predictions	Provides predictive analysis results

The Flask backend acts as the communication layer between the frontend dashboard and the underlying processing and control modules.

4.9 DATA LOGGING DESIGN

The system uses lightweight CSV-based logging for recording operational information. During system operation, relevant information such as frame identification numbers, timestamps, person counts, density estimates, device status indicators, and tracking IDs can be stored for subsequent analysis.

TABLE 4.8: LOGGING STRUCTURE

Field	Description
Frame ID	Identification number of the processed frame
Timestamp	Time of observation
Person Count	Number of detected individuals
Density	Estimated crowd density
Temperature	Current temperature
Humidity	Current humidity
Tracking IDs	Identifiers assigned to tracked persons
Device Status	Current environmental device state

4.10 UML DIAGRAMS

Unified Modeling Language (UML) diagrams were meticulously constructed to document the system's dynamic behaviors and structural relationships, ensuring all actors and use cases were properly mapped before implementation commenced.

- ❖ Use Case Diagram
- ❖ Sequence Diagram
- ❖ Activity Diagram

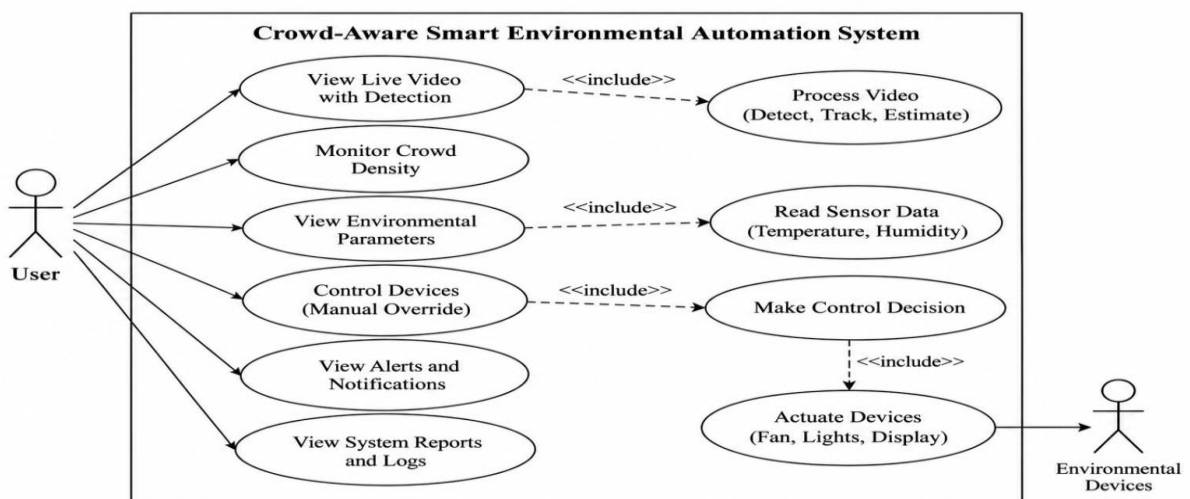


Figure 4.3 UML Use Case Diagram

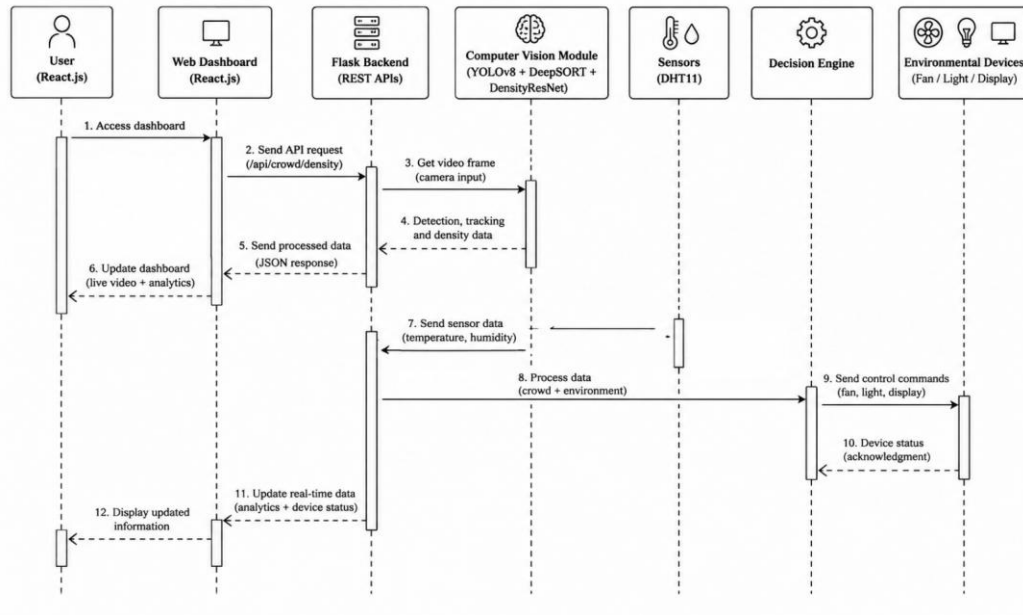


Figure 4.4 UML Sequence Diagram

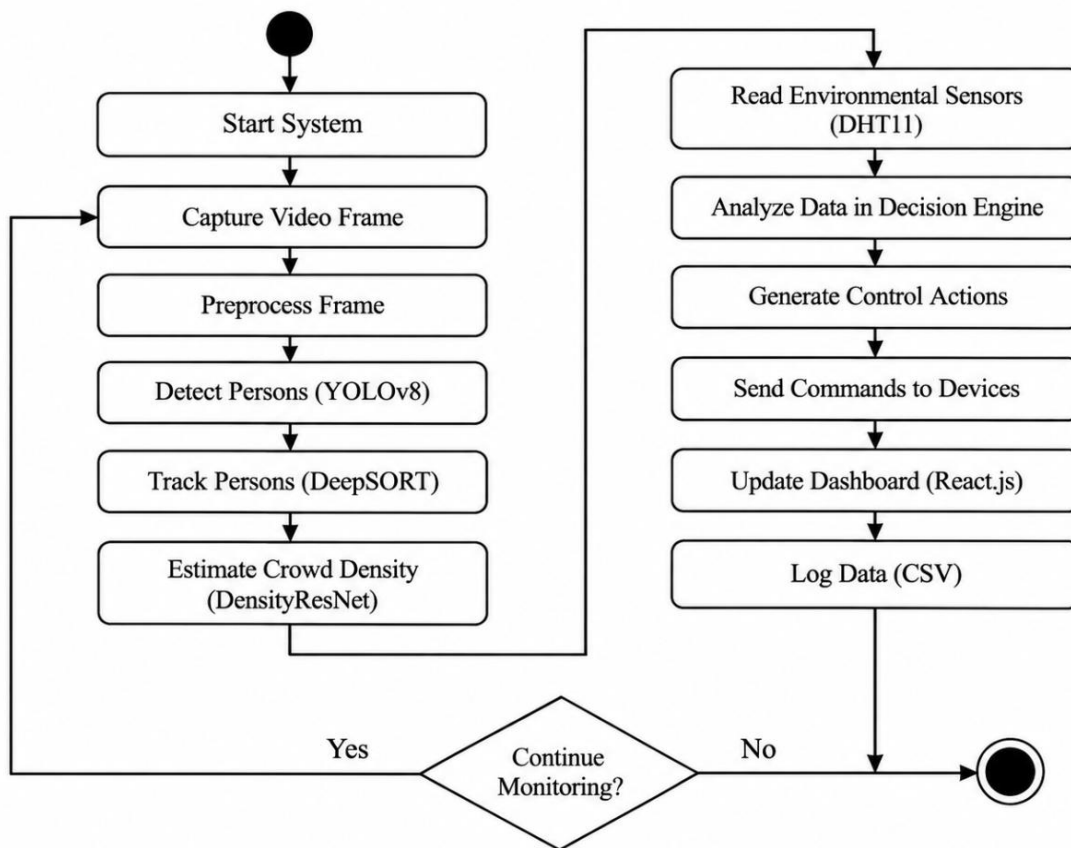


Figure 4.5 UML Activity Diagram

4.11 SUMMARY

This chapter presented the detailed design of the proposed crowd-aware environmental automation system. The architecture integrates YOLOv8-based crowd detection, Multi-Scale DensityResNet-based density estimation, DeepSORT-based tracking and re-identification, intelligent decision-making, and adaptive environmental control. The architecture and component-level designs provide the foundation for implementing the complete system.

CHAPTER 5

IMPLEMENTATION

5.1 SYSTEM SPECIFICATION

The proposed system was implemented as an integrated computer vision, machine learning, IoT, backend, and frontend application. The computer vision components were developed using Python-based deep learning and image-processing libraries, while the backend was implemented using Flask REST APIs. The user interface was developed using React.js. The system architecture was designed for edge-based processing using Raspberry Pi 4, camera input, DHT11 environmental sensors, and connected control devices.

TABLE 5.1: SYSTEM IMPLEMENTATION SPECIFICATION

Component	Specification
Edge Platform	Raspberry Pi 4
Camera	Real-time camera input
Environmental Sensor	DHT11
Person Detection	Enhanced YOLOv8-Crowd
Crowd Density	Multi-Scale DensityResNet
Tracking	Enhanced DeepSORT
Re-Identification	EfficientNet-B2
Backend	Python Flask
Frontend	React.js
Data Logging	CSV
Communication	REST API / GPIO
Primary Dataset	ShanghaiTech
Auxiliary Datasets	UCF-QNRF, JHU-CROWD++, Mall, UCSD

5.2 DATASET PREPARATION AND PREPROCESSING

The implementation uses ShanghaiTech as the primary crowd-counting dataset. The dataset consists of 1,198 high-resolution images divided into Part A and Part B. Part A contains 482 images, with 300 training images and 182 testing images. Part B contains 716 images, with 400 training images and 316 testing images.

The enhanced annotations contain 15,212 individual person annotations, including additional information such as occlusion flags, pose variations, and demographic attributes.

The auxiliary datasets specified in the report include:

- UCF-QNRF — 1,535 images and 1.25M annotated heads.
- JHU-CROWD++ — 4,372 images and 1.51M annotations.
- Mall Dataset — 2,000 frames.
- UCSD Dataset — 2,000 frames

TABLE 5.2: PREPROCESSING PARAMETERS

Preprocessing Operation	Configuration
Target Image Size	640×640
Horizontal Flip	$p = 0.5$
Rotation	$\pm 15^\circ, p = 0.3$
Perspective Transform	distortion = 0.1, $p = 0.4$
MixUp	$\alpha = 0.2$
Gaussian Blur	Kernel 3–7, $p = 0.2$
Cutout	16×16 patches, $p = 0.3$
Mosaic	Used for multi-scale training
Copy-Paste	Used for minority class balancing

5.3 CROWD DETECTION IMPLEMENTATION

The crowd detection module is implemented using the enhanced YOLOv8-Crowd model. The prepared ShanghaiTech dataset is loaded and images are resized to 640×640 pixels. Data augmentation techniques such as flipping, rotation, perspective transformation, MixUp, Mosaic, and Copy-Paste are applied to improve model generalization.

The modified YOLOv8 architecture incorporates enhanced CSP modules, attention mechanisms, SPP-ELAN, and hybrid PAN/FPN feature fusion for improved multi-scale crowd detection. Soft-NMS is used during post-processing to preserve valid detections in highly overlapping crowd regions.

The detection model is trained using the configuration presented in Table 4.1. During inference, the model produces bounding boxes and confidence scores for detected persons. These detections are then passed to the DeepSORT tracking module for identity tracking.

Implementation flow:

Dataset $\rightarrow$ Preprocessing $\rightarrow$ Augmentation $\rightarrow$ YOLOv8 Detection $\rightarrow$ Soft-NMS $\rightarrow$ Bounding Boxes $\rightarrow$ DeepSORT

5.4 CROWD DENSITY ESTIMATION IMPLEMENTATION

The crowd density estimation module is implemented using the Multi-Scale DensityResNet architecture defined in Table 4.2. The input image is first passed through the stem convolution layer and ResBlock stages to extract features at different scales. The architecture uses stride-based downsampling and dilated convolutions to capture both spatial and contextual information from crowded scenes.

The extracted features are processed using the Atrous Spatial Pyramid Pooling (ASPP) module with multiple dilation rates to capture multi-scale contextual information. The CBAM attention module then refines the feature representation by focusing on important channel and spatial information. Adaptive Gaussian kernels are used to generate the ground-truth density maps from annotated head positions.

Finally, the refined features are passed through the progressive upsampling decoder to reconstruct the final single-channel density map. Skip connections preserve fine-grained spatial details during reconstruction and improve gradient flow.

The overall implementation follows the sequence Input Image $\rightarrow$ Stem Convolution $\rightarrow$ ResBlocks $\rightarrow$ ASPP $\rightarrow$ CBAM $\rightarrow$ Progressive Decoder $\rightarrow$ Density Map, using the architecture specified in Table 4.2.

5.5 DEEPSORT TRACKING IMPLEMENTATION

The person tracking module is implemented using the DeepSORT tracking algorithm integrated with the YOLOv8 detection module. The tracking pipeline begins with YOLOv8 detecting individuals and generating bounding boxes. These bounding boxes are then passed to the tracking module, where the adaptive Kalman filter predicts the movement of existing tracks. Appearance features are extracted using the EfficientNet-B2 re-identification network to help maintain person identity across consecutive frames.

The re-identification network generates 256-dimensional L2-normalized feature vectors. It uses a triplet loss margin of 0.3 and a center loss weight of 0.003 to learn discriminative appearance features. The adaptive Kalman filter adjusts its

parameters according to crowd density and occlusion conditions, improving tracking stability in crowded scenes.

Data association combines IoU, appearance, and motion information using a weighted cost function:

$$\text{Cost} = w_1 \text{IoUcost} + w_2 \text{Appearancecost} + w_3 \text{Motioncost}$$

where $w_1 = 0.4$, $w_2 = 0.4$, and $w_3 = 0.2$. The final association results are used to assign and maintain unique **Track IDs** for individuals across frames. Thus, the complete implementation pipeline is

YOLOv8 Detection → Bounding Boxes → Kalman Prediction → Appearance Feature Extraction → Data Association → Track ID.

5.6 IOT AND SENSOR IMPLEMENTATION

The environmental sensing module is implemented using DHT11 sensors connected to the Raspberry Pi through GPIO communication. The sensors continuously monitor the environmental conditions of the monitored area and provide temperature and humidity readings to the Python-based system. The DHT11 specifications used in the implementation are given in Table 4.4.

The temperature and humidity values are periodically read from the DHT11 sensor at the specified sampling interval. The sensor readings are transferred through the Raspberry Pi GPIO interface and processed by the Python/Flask backend. The collected values are then forwarded to the decision engine along with the crowd density information obtained from the computer vision pipeline.

The decision engine uses the combined crowd density, temperature, and humidity values to determine the required environmental response. Based on the

calculated control conditions, appropriate commands are generated for connected environmental devices such as fans and lighting systems.

The complete implementation flow is DHT11 → Raspberry Pi GPIO → Sensor Reading → Flask/Python Processing → Decision Engine → Device Control.

5.7 DECISION ENGINE IMPLEMENTATION

The decision engine is implemented to convert the crowd and environmental information into appropriate control commands. It receives crowd count, crowd density, temperature, humidity, context, and time information as inputs. These inputs are processed using the dynamic threshold and control equations defined in Section 4.6.

The implementation first reads the current crowd density and environmental sensor values. Based on the operating context and time period, the corresponding context factor and time factor are determined. These values are then used to calculate the dynamic threshold according to the equation defined in the system design. Finally, the calculated control values are used to generate commands for the connected environmental devices.

The implementation follows the sequence Input Data → Context and Time Analysis → Dynamic Threshold Calculation → Control Value Calculation → Device Command, enabling the system to adapt fan, lighting, and temperature control according to the current conditions.

5.8 REACT.JS FRONTEND IMPLEMENTATION

The frontend of the proposed system is implemented using React.js to provide an interactive dashboard for monitoring crowd and environmental conditions. The

dashboard displays live crowd information, crowd density, temperature, humidity, and connected device status in an organized interface.

The live video module visualizes the video stream along with the crowd detection information generated by the computer vision pipeline. The device control section displays the status of environmental devices and provides user interaction with controls where implemented. The interface also provides alerts and threshold-based notifications to indicate important changes or system conditions.

The React.js frontend communicates with the Flask REST backend to retrieve and display system data. API communication is used to obtain crowd information, environmental sensor readings, analytics, and device status from the backend. The overall frontend flow is Flask Backend → REST API → React.js Components → Dashboard Visualization, providing a centralized interface for real-time system monitoring and control.

5.9 FLASK BACKEND IMPLEMENTATION

TABLE 5.3: MAIN API STRUCTURE

Endpoint	Implementation Purpose
/api/crowd/density	Returns crowd/density information
/api/environment/sensors	Returns temperature/humidity data
/api/control/devices	Handles device status/control
/api/analytics/predictions	Returns analytical/prediction information

The Flask backend receives requests from the React.js frontend, processes or retrieves the required information, and returns structured responses. The backend

also provides the interface between the web dashboard and the edge-side processing components.

5.10 SUMMARY

This chapter described the implementation of the proposed crowd-aware environmental automation system. The implementation integrates the prepared crowd datasets, enhanced YOLOv8 detection, Multi-Scale DensityResNet density estimation, DeepSORT tracking, intelligent decision logic, React.js frontend, Flask backend, and CSV-based logging. The individual components were implemented as interconnected modules to provide a complete real-time monitoring and environmental-control pipeline.

CHAPTER 6

RESULTS AND DISCUSSION

6.1 SYSTEM RESULTS

The proposed system was evaluated across crowd detection, density estimation, person tracking, environmental monitoring, and real-time system performance. The reported results demonstrate the operation of the integrated computer vision, IoT, machine learning, and environmental control framework.

The proposed method reported 91.2% crowd detection accuracy, 94.2% mAP, 91.8% identity preservation, 45 FPS effective processing, and 22 ms total latency. The system also reported a 28.5% reduction in energy consumption while maintaining environmental comfort within $\pm 0.5^{\circ}\text{C}$ of the target setpoint.

These results indicate the performance of the integrated system across both computer vision accuracy and real-time environmental automation.

6.2 CROWD DETECTION RESULTS

The crowd detection module was evaluated using the enhanced YOLOv8-Crowd architecture. The reported detection performance achieved 91.2% detection accuracy and 94.2% mean Average Precision. The system was designed to handle crowded scenes containing overlapping and partially occluded persons.

The occlusion evaluation further measured detection accuracy at different levels of occlusion. The proposed method achieved 94.8% at low occlusion, 89.3% at medium occlusion, 78.6% at high occlusion, and 62.4% at severe occlusion, with an average performance of 81.3%.

TABLE 6.1: OCCLUSION HANDLING EVALUATION

Occlusion Level	Proposed Method	ResNet Baseline	OCRNet	CSRNet
Low Occlusion (0–30%)	94.8	87.2	89.1	88.4
Medium Occlusion (30–60%)	89.3	76.5	79.8	78.2
High Occlusion (60–80%)	78.6	58.3	63.7	61.9
Severe Occlusion (>80%)	62.4	31.8	38.2	35.6
Average Performance	81.3	63.5	67.7	66.0

6.3 CROWD DENSITY ESTIMATION RESULTS

The Multi-Scale DensityResNet was evaluated for its ability to estimate crowd density from input images. The proposed method reported MAE values of 6.9 for Part A and 2.1 for Part B, with a reported MAPE of 4.8%. The density estimation results were also evaluated using PSNR, SSIM, correlation coefficient, and runtime.

The reported comparison showed a PSNR of 32.4 dB, SSIM of 0.89, and correlation coefficient of 0.94 for the proposed method. The reported runtime for the density estimation comparison was 22 ms.

Density Estimation Performance Comparison:

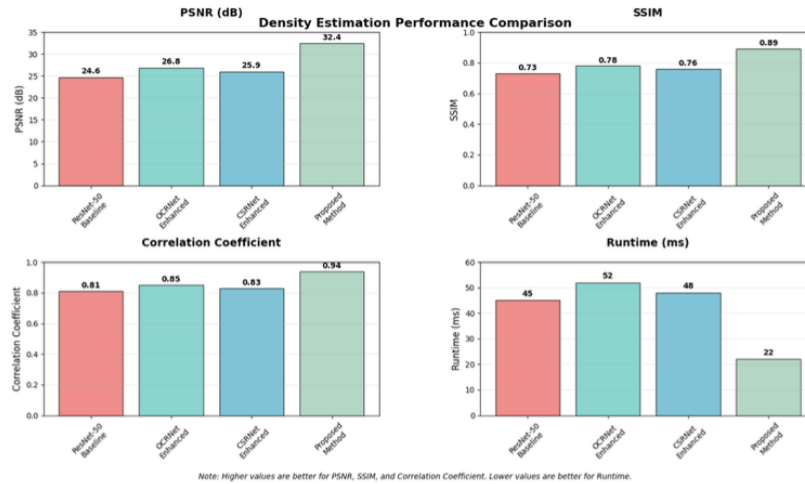


Figure 6.1 – Density Estimation Performance Comparison

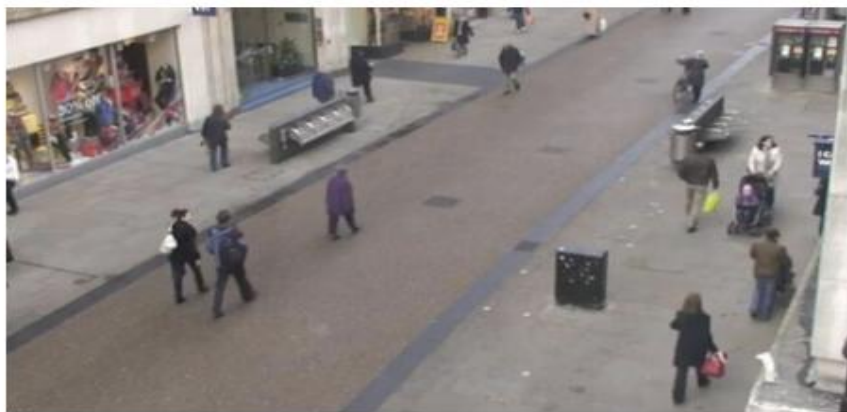


Figure 6.2 CCTV Captured Crowded Image



Figure 6.3 – Calculated Density Using Proposed System

6.4 PERSON TRACKING RESULTS

The person tracking module was implemented using DeepSORT with the enhanced re-identification mechanism based on EfficientNet-B2. The system uses appearance features, Kalman filtering, and data association to maintain consistent identities across successive frames.

The reported identity preservation performance was **91.8%**, particularly considering occlusion and viewpoint changes. The occlusion evaluation also demonstrates the effect of the enhanced tracking and re-identification mechanism on maintaining detection performance under different crowd conditions.

6.5 ENVIRONMENTAL MONITORING RESULTS

The environmental monitoring module collects temperature and humidity information using the DHT11 sensors connected through the Raspberry Pi GPIO interface. The collected sensor information is passed to the processing system together with the crowd information obtained from the computer vision pipeline.

The environmental information is used by the decision engine to determine the appropriate response under changing crowd and environmental conditions. The reported system maintained environmental comfort within $\pm 0.5^{\circ}\text{C}$ of the target setpoints during the evaluated operation.

The environmental monitoring information is also made available through the React.js dashboard, where temperature and humidity values can be viewed along with the current crowd and device information.

No numerical temperature/humidity chart is present in the images you provided, so do not create one unless you have actual recorded sensor data.

6.6 ENVIRONMENTAL CONTROL RESULTS

The environmental control module uses the output of the decision engine to determine appropriate control actions for connected devices. The control process considers crowd density together with environmental conditions and generates commands for devices such as fans and lighting systems.

The reported evaluation showed a 28.5% reduction in energy consumption while maintaining the required environmental comfort conditions. This demonstrates the integration between crowd analysis, environmental sensing, decision-making, and automated control.

The environmental control output forms the final stage of the system pipeline, converting the detected crowd and environmental conditions into adaptive device responses.

6.7 REACT.JS DASHBOARD OUTPUT

The React.js frontend provides a centralized interface for monitoring the complete system. The dashboard displays live crowd information, crowd density, temperature, humidity, device status, live video information, and system alerts.

The live video section presents the video stream together with the crowd detection information generated by the computer vision pipeline. Environmental monitoring components display temperature and humidity information, while device status components provide the current state of connected environmental devices.

The React.js frontend communicates with the Flask REST backend to retrieve crowd, sensor, analytics, and device information. This provides a unified interface for observing the real-time operation of the proposed system.

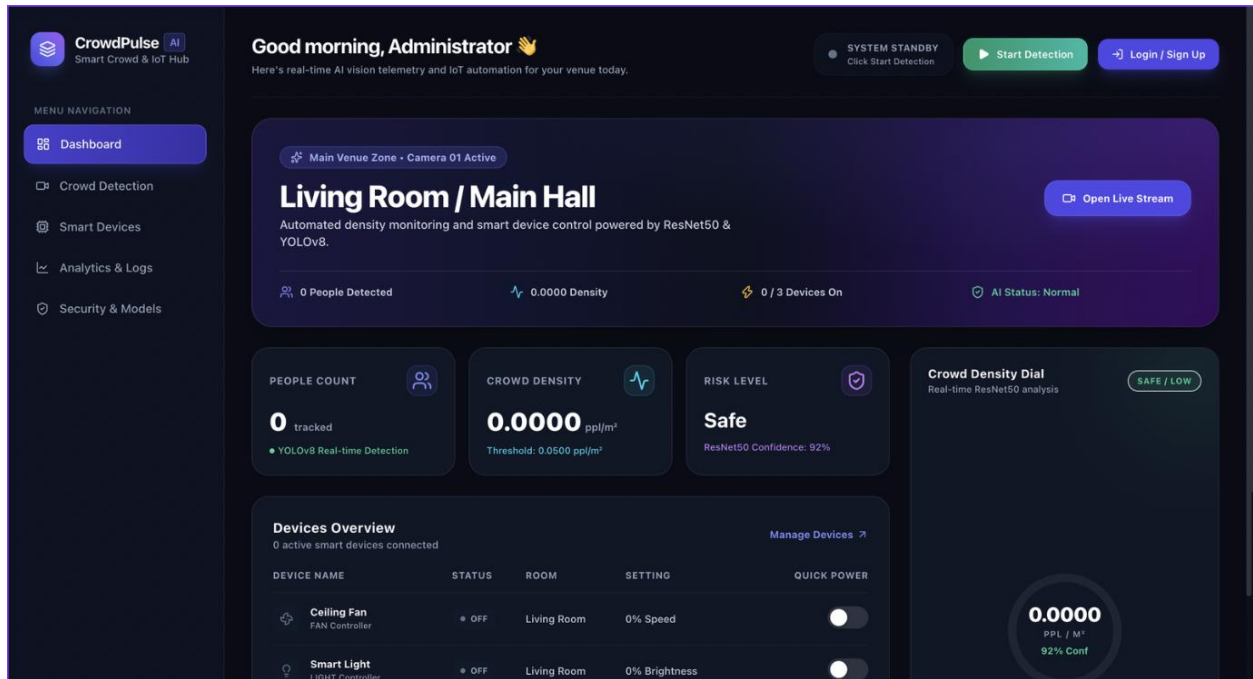


Figure 6.4 React.js Dashboard Output

6.8 PERFORMANCE ANALYSIS

The real-time processing performance was evaluated by measuring the latency of each stage of the processing pipeline. The reported stages include frame capture, preprocessing, YOLOv8 detection, DeepSORT tracking, density estimation, decision making, and device control.

TABLE 6.2: REAL-TIME PROCESSING PIPELINE PERFORMANCE

Processing Stage	Latency (ms)	Optimization
Frame Capture	33	30 FPS capture
Preprocessing	3	Parallel processing
YOLOv8 Detection	8	GPU acceleration
DeepSORT Tracking	4	Optimized Hungarian
Density Estimation	5	Quantized model
Decision Making	1	Rule-based engine
Device Control	1	GPIO/API calls
Total Latency	22	45 FPS effective

The reported total latency is 22 ms, with an effective processing performance of 45 FPS. The individual stages demonstrate that the detection, tracking, density estimation, decision-making, and device-control components contribute to the complete real-time processing pipeline.

6.9 COMPARATIVE ANALYSIS

The proposed method was compared with three configurations using MAE, MAPE, FPS, parameter count, and [mAP@0.5](#). The comparison considers both crowd estimation accuracy and computational performance.

TABLE 6.3: COMPARATIVE PERFORMANCE ANALYSIS

Method	MAE (Part A)	MAE (Part B)	MAPE (%)	FPS	Parameters (M)	mAP@0.5
ResNet-50 + DeepSORT + YOLOv8	45.7	8.9	18.4	25	43.2	0.78
OCRNet + Enhanced DeepSORT + YOLOv8	38.2	7.1	15.6	20	51.8	0.82
CSRNet + Enhanced DeepSORT + YOLOv8	42.1	8.3	17.1	22	48.7	0.80
Proposed Method	6.9	2.1	4.8	45	18.3	0.94

The proposed configuration reports lower MAE and MAPE values together with higher FPS and [mAP@0.5](#) than the configurations shown in the comparison. The reported parameter count is also lower than the other configurations in the table.

6.10 DISCUSSION

The evaluation results show that the proposed system combines crowd detection, density estimation, person tracking, environmental sensing, and adaptive control within a single framework. The reported computer vision results demonstrate performance under different levels of crowding and occlusion, while the density estimation results provide quantitative measurements using MAE, PSNR, SSIM, and correlation coefficient. The ablation results also provide evidence regarding the contribution of the architectural components. The CBAM configuration reported 91.2% detection accuracy, while the full multi-task loss configuration reported an MAE of 6.9 and detection mAP of 0.94.

CHAPTER 7

CONCLUSION AND FUTURE ENHANCEMENTS

7.1 CONCLUSION

The proposed Real-Time Crowd-Aware Smart Environmental Automation System integrates computer vision, machine learning, IoT-based environmental sensing, and adaptive control into a unified framework. The system combines YOLOv8-Crowd for crowd detection, Multi-Scale DensityResNet for density estimation, enhanced DeepSORT for person tracking, DHT11 sensors for environmental monitoring, a Flask backend, and a React.js frontend to support real-time crowd-aware environmental management.

The reported evaluation results demonstrate the performance of the proposed framework across different system components. The system achieved a reported MAE of 6.9, 94.2% mAP, 91.8% identity preservation, 22 ms total processing latency, and 45 FPS effective performance. The system also reported a 28.5% reduction in energy consumption while maintaining environmental comfort within $\pm 0.5^{\circ}\text{C}$ of the target setpoints.

The integration of crowd information with temperature and humidity measurements enables the decision engine to generate adaptive environmental control commands. The React.js dashboard provides a centralized interface for monitoring crowd conditions, environmental parameters, device status, live video information, and system alerts. Overall, the implemented framework demonstrates the integration of computer vision, IoT sensing, and automated environmental control for crowd-aware smart environments.

7.2 LIMITATIONS

The proposed system has certain limitations related to computational resources, sensor accuracy, dataset diversity, and deployment conditions. The use of multiple computer vision stages, including detection, density estimation, and tracking, can require considerable processing resources. The DHT11 sensor also has limitations in measurement accuracy and operating range. In addition, the performance of the system may vary under different camera configurations, crowd distributions, environmental conditions, and real-world deployment scenarios.

7.3 FUTURE ENHANCEMENTS

Future enhancements can focus on improving the scalability, intelligence, real-time performance, and IoT capabilities of the system. The proposed framework can be extended with full-scale IoT implementation by integrating additional environmental sensors, actuators, and connected devices for comprehensive real-time monitoring and automated control. Federated learning can be used for collaborative model improvement, while 5G network optimization can support low-latency communication. Transformer-based architectures can improve temporal modeling of crowd dynamics, and advanced predictive analytics can be incorporated to predict future crowd and environmental conditions. Further research can also explore quantum computing integration and advanced optimization techniques for large-scale smart-environment applications..



SUSTAINABLE DEVELOPMENT GOALS MAPPING

The project Real-Time Crowd-Aware Smart Environmental Automation System primarily contributes to SDG 7 – Affordable and Clean Energy, SDG 9 – Industry, Innovation and Infrastructure, SDG 11 – Sustainable Cities and Communities, SDG 12 – Responsible Consumption and Production, and SDG 13 – Climate Action.

Questions	Answer Samples
Which SDGs does the project directly address?	SDG 7 – Affordable and Clean Energy, SDG 9 – Industry, Innovation and Infrastructure, SDG 11 – Sustainable Cities and Communities, SDG 12 – Responsible Consumption and Production, and SDG 13 – Climate Action.
What strategies or actions are being implemented to achieve these goals?	The system uses real-time crowd detection, density estimation, IoT-based temperature and humidity monitoring, and adaptive environmental control to adjust devices such as fans and lighting according to actual crowd and environmental conditions.
How is progress measured and reported in relation to the SDGs?	Progress is evaluated using system performance and environmental-

	control metrics, including the reported 28.5% reduction in energy consumption, 22 ms processing latency, 45 FPS effective performance, and environmental comfort maintained within $\pm 0.5^{\circ}\text{C}$ of target setpoints.
How were these goals identified as relevant to the project's objectives?	These goals were identified because the project's main objectives are to improve energy efficiency, intelligent automation, resource utilization, and sustainable environmental management through the integration of computer vision, machine learning, IoT, and real-time control.
Are there any partnerships or collaborations in place to enhance this impact?	The current project does not specify any formal external partnerships or collaborations. Future deployment could involve collaboration with educational institutions, healthcare facilities, commercial buildings, and smart-infrastructure organizations to evaluate and expand the system in real-world environments.

REFERENCES

- [1] P. W. Tien, S. Wei, and J. Calautit, “A computer vision-based occupancy and equipment usage detection approach for reducing building energy demand,” *Energies*, vol. 14, no. 1, p. 156, Dec. 2020. DOI: 10.3390/en14010156.
- [2] S. M. Patankar, R. K. Swami, and S. Nanivadekar, “Enhancing energy efficiency in commercial office buildings: A smart IoT and machine vision approach,” in *Proc. 2nd World Conf. Communication & Computing (WCONF)*, Raipur, India, 2024, pp. 1–5. DOI: 10.1109/WCONF61366.2024.10691999.
- [3] B. S. Kumar, S. Ramalingam, S. Viveka, V. G. Vigneshwar, N. Sivasubramaniam, and J. S. Maria, “Intelligent automation system for energy conservation using IoT and machine learning,” in *Proc. 7th Int. Conf. Electronics, Communication and Aerospace Technology (ICECA)*, Coimbatore, India, 2023, pp. 1841–1846. DOI: 10.1109/ICECA58529.2023.10395008.
- [4] Y. Li, X. Zhang, and D. Chen, “CSRNet: Dilated convolutional neural networks for understanding the highly congested scenes,” *arXiv preprint arXiv:1802.10062*, 2018. DOI: 10.48550/arXiv.1802.10062.
- [5] X. Wang and W. Jia, “Optimizing Edge AI: A comprehensive survey on data, model, and system strategies,” *Qeios*, Jan. 2025. DOI: 10.32388/IZOHCH.
- [6] T. M. Sanjeev Kumar, S. G. Varghese, C. P. Kurian, and C. Mouli, “Low-cost image-based occupancy sensor using deep learning,” in *Advances in Renewable Energy and Electric Vehicles*, P. S., N. Prabhu, and K. S., Eds., *Lecture Notes in Electrical Engineering*, vol. 767. Singapore: Springer, 2022. DOI: 10.1007/978-981-16-1642-6_22.

- [7] V. Shankar, “Edge AI: A comprehensive survey of technologies, applications, and challenges,” ResearchGate, 2024. [Online]. Available: <https://www.researchgate.net/publication/385370366>
- [8] S. Wei, P. W. Tien, and J. K. Calautit, “The impact of deep learning-based equipment usage detection on building energy demand estimation,” *Building Services Engineering Research and Technology*, vol. 42, no. 5, pp. 551–563, 2021. DOI: 10.1177/01436244211034737.
- [9] B. Li, H. Huang, A. Zhang, et al., “Approaches on crowd counting and density estimation: A review,” *Pattern Analysis and Applications*, vol. 24, pp. 853–874, Aug. 2021. DOI: 10.1007/s10044-021-00959-z.
- [10] S. Lee, S. H. Nengroo, H. Jin, Y. Doh, C. Lee, T. Heo, and D. Har, “Power management in smart residential building with deep learning model for occupancy detection by usage pattern of electric appliances,” *arXiv preprint arXiv:2209.11520*, Sep. 2022. DOI: 10.48550/arXiv.2209.11520.
- [11] Ahamed, C. D. Ranathunga, D. S. Udayantha, B. K. K. Ng, and C. Yuen, “Real-time AI-driven people tracking and counting using overhead cameras,” *arXiv preprint arXiv:2411.10072*, Nov. 2024. DOI: 10.48550/arXiv.2411.10072.
- [12] T. M. Sanjeev Kumar, S. G. Varghese, C. P. Kurian, and C. Mouli, “Low-cost image-based occupancy sensor using deep learning,” in *Advances in Renewable Energy and Electric Vehicles*, P. S., N. Prabhu, and K. S., Eds., *Lecture Notes in Electrical Engineering*, vol. 767. Singapore: Springer, 2022. DOI: 10.1007/978-981-16-1642-6_22.

- [13] P. Galluzzi, E. Longo, A. E. C. Redondi, and M. Cesana, “Occupancy estimation using low-cost Wi-Fi sniffers,” arXiv preprint arXiv:1905.06809, May 2019. DOI: 10.48550/arXiv.1905.06809.
- [14] J. Khan, M. Fayaz, U. Zaman, K. Kim, et al., “A hybrid machine learning and optimization algorithm for enhanced user comfort and energy efficiency in smart homes,” Preprints, Jan. 2024. DOI: 10.20944/preprints202401.1331.v1.
- [15] A. N. Sayed, F. Bensaali, Y. Himeur, and M. Houchati, “Leveraging machine learning for identifying occupancy patterns from power data with a moving window feature extraction method,” in Proc. 9th Int. Congress on Information and Communication Technology (ICICT 2024), Lecture Notes in Networks and Systems, vol. 1003, Singapore: Springer, 2024, pp. 161–171. DOI: 10.1007/978-981-97-3302-6_14.
- [16] M. Esrafilian-Najafabadi and F. Haghighat, “Occupancy-based HVAC control using deep learning algorithms for estimating online preconditioning time in residential buildings,” *Energy and Buildings*, vol. 252, p. 111377, Aug. 2021. DOI: 10.1016/j.enbuild.2021.111377.
- [17] A. N. Sayed, F. Bensaali, Y. Himeur, and M. Houchati, “Edge-based real-time occupancy detection system through a non-intrusive sensing system,” *Energies*, vol. 16, no. 5, p. 2388, 2023. DOI: 10.3390/en16052388.
- [18] J.-A. Jiang, J.-C. Wang, H.-S. Wu, Y.-C. Yang, et al., “A novel sensor placement strategy for an IoT-based power grid monitoring system,” *IEEE Internet of Things Journal*, early access, Apr. 2020. DOI: 10.1109/JIOT.2020.2991610.

- [19] T. Vafeiadis, et al., “Machine learning-based occupancy detection via the use of smart meters,” in Proc. Int. Symp. Computer Science and Intelligent Controls (ISCSIC), Budapest, Hungary, 2017, pp. 6–12. DOI: 10.1109/ISCSIC.2017.15.
- [20] Z. Luo, R. Qi, Q. Li, J. Zheng, and S. Shao, “ABODE-Net: Attention-based deep learning for occupancy detection using smart meter data,” in Springer Lecture Notes in Computer Science. DOI: 10.1007/978-3-031-28124-2_15.
- [21] X. Liang and H. Wang, “Hybrid Transformer-RNN architecture for household occupancy detection,” arXiv preprint arXiv:2308.14114, 2023. DOI: 10.48550/arXiv.2308.14114.
- [22] J. Soldano, “Sensor comparison: DHT11 vs DHT22 vs BME680 vs DS18B20,” Instructables, [Online]. Available: <https://www.instructables.com/Sensor-Comparison-DHT11-Vs-DHT22-Vs-BME680-Vs-DS18/>
- [23] D. Zhang, “Research on HVAC occupancy detection with ML and DL methods,” Atlantis Press. DOI: 10.2991/978-94-6463-512-6_69.
- [24] T. Moraes, B. C. S. Nogueira, V. Lira, and E. Tavares, “Performance comparison of IoT communication protocols,” in Proc. 2019 IEEE Int. Conf. Systems, Man and Cybernetics (SMC), Bari, Italy, Oct. 2019, pp. 3708–3713. DOI: 10.1109/SMC.2019.8914552.
- [25] M. A. Abuhussain, B. S. Alotaibi, Y. A. Dodo, A. Maghrabi, and M. S. Aliero, “Multimodal framework for smart building occupancy detection,” Sustainability. DOI: 10.3390/su16104171.
- [26] M. Esrafilian-Najafabadi and F. Haghighat, “Towards self-learning control of HVAC systems with the consideration of dynamic occupancy patterns:

APPENDICES

APPENDIX 1 – SELECTED SOURCE CODE

Seg12.py //Models and Core Logic

```
# ----- SYSTEM INITIALIZATION -----  
  
app = Flask(__name__)  
CORS(app)  
CAMERA_ID = 0  
CAMERA_FOV_M2 = 100  
DENSITY_THRESHOLD = 0.05  
MIN_CONFIDENCE = 0.3  
CSV_FILE = "density_log.csv"  
device = (  
    torch.device("mps")  
    if torch.backends.mps.is_available()  
    else torch.device("cpu")  
)  
# YOLOv8 person detection model  
yolo_model = YOLO("yolov8n.pt")  
if device.type == "mps":  
    yolo_model.to(device)  
# ----- DENSITY RESNET -----  
  
class DensityResNet(nn.Module):  
    def __init__(self):  
        super().__init__()
```

```

self.base_model = models.resnet50(
    weights="IMAGENET1K_V2"
)
self.base_model.fc = nn.Identity()
self.base_model.avgpool = nn.Identity()

self.density_head = nn.Sequential(
    nn.Conv2d(2048, 1024, 3, padding=1),
    nn.ReLU(inplace=True),
    nn.Conv2d(1024, 512, 3, padding=1),
    nn.ReLU(inplace=True),

    nn.Conv2d(512, 256, 3, padding=1),
    nn.ReLU(inplace=True),
    nn.Conv2d(256, 1, 1),
    nn.ReLU()
)

def forward(self, x):
    x = self.base_model.conv1(x)
    x = self.base_model.bn1(x)
    x = self.base_model.relu(x)
    x = self.base_model.maxpool(x)
    x = self.base_model.layer1(x)
    x = self.base_model.layer2(x)
    x = self.base_model.layer3(x)

```

```

x = self.base_model.layer4(x)
x = self.density_head(x)
return nn.functional.interpolate(
    x,
    size=(224, 224),
    mode="bilinear",
    align_corners=False
)

# ----- DEEPSORT TRACKER -----
tracker = DeepSort(
    max_age=50,
    n_init=3,
    max_cosine_distance=0.4,
    nn_budget=None,
    embedder="mobilenet",
    half=True,
    bgr=True,
    embedder_gpu=(device.type == "mps")
)

# ----- CROWD DENSITY ANALYSIS -----
def analyze_density(frame):
    frame_rgb = cv2.cvtColor(
        frame,
        cv2.COLOR_BGR2RGB
    )

```

```

input_tensor = transform(
    frame_rgb
).unsqueeze(0).to(device)
with torch.no_grad():
    density_map = density_model(
        input_tensor
    )
    crowd_count = torch.sum(
        density_map
    ).item()
return crowd_count

# ----- MAIN PROCESSING PIPELINE -----
def process_frame(frame):
    start_time = time.time()
    # YOLOv8 detection
    results = yolo_model(
        frame,
        verbose=False,
        conf=MIN_CONFIDENCE
    )
    detections = []
    for result in results:
        for box in result.bboxes:
            class_id = int(box.cls[0])
            confidence = float(box.conf[0])

```

```

if yolo_model.names[class_id] == "person":
    x1, y1, x2, y2 = map(
        int,
        box.xyxy[0]
    )
    detections.append(
        (
            [x1, y1, x2, y2],
            confidence,
            0
        )
    )

```

DeepSORT tracking

```

tracks = tracker.update_tracks(
    detections,
    frame=frame
)
tracked_ids = []
tracked_people = []
for track in tracks:
    if not track.is_confirmed():
        continue
    track_id = track.track_id
    x1, y1, x2, y2 = map(

```

```

        int,
        track.to_ltrb()
    )
    tracked_ids.append(track_id)
    tracked_people.append(
        (x1, y1, x2, y2, track_id)
    )
# Crowd count and density
    people_count = len(tracked_people)
    density = (
        people_count /
        CAMERA_FOV_M2
    )
# DensityResNet analysis
    estimated_count = analyze_density(
        frame
    )

```

APPENDIX 2 – API ENDPOINT SUMMARY

The Flask backend provides REST API endpoints for communication between the **React.js frontend** and the system modules. The APIs allow the frontend to retrieve crowd information, environmental sensor readings, analytics, and device-control information.

S. No.	API Endpoint	Method	Purpose
1	/api/crowd/density	GET	Retrieves current crowd count and crowd density information.
2	/api/environment/sensors	GET	Retrieves temperature and humidity readings from the environmental sensors.
3	/api/control/devices	GET / POST	Retrieves device status and handles device-control information.
4	/api/analytics/predictions	GET	Retrieves crowd and environmental analytics or prediction information.

APPENDIX 3 – DATA LOGGING STRUCTURE

The system uses CSV-based logging to store relevant crowd and environmental monitoring information. The logged data can include crowd count, crowd density, temperature, humidity, timestamps, and device/control status. The CSV-based approach provides a simple mechanism for recording system observations and supporting later analysis.

Sample Data Logging Structure

Field	Description
Timestamp	Date and time of the recorded observation
Crowd Count	Number of detected persons
Crowd Density	Estimated crowd density
Temperature	Temperature recorded by the DHT11 sensor
Humidity	Humidity recorded by the DHT11 sensor
Device Status	Current status of connected environmental devices
Control Action	Action generated by the decision engine

Data Flow:

Sensors + Computer Vision → Flask/Python Processing → Decision Engine → CSV Data Logging → React.js Dashboard

APPENDIX 4 – ADDITIONAL SYSTEM OUTPUTS

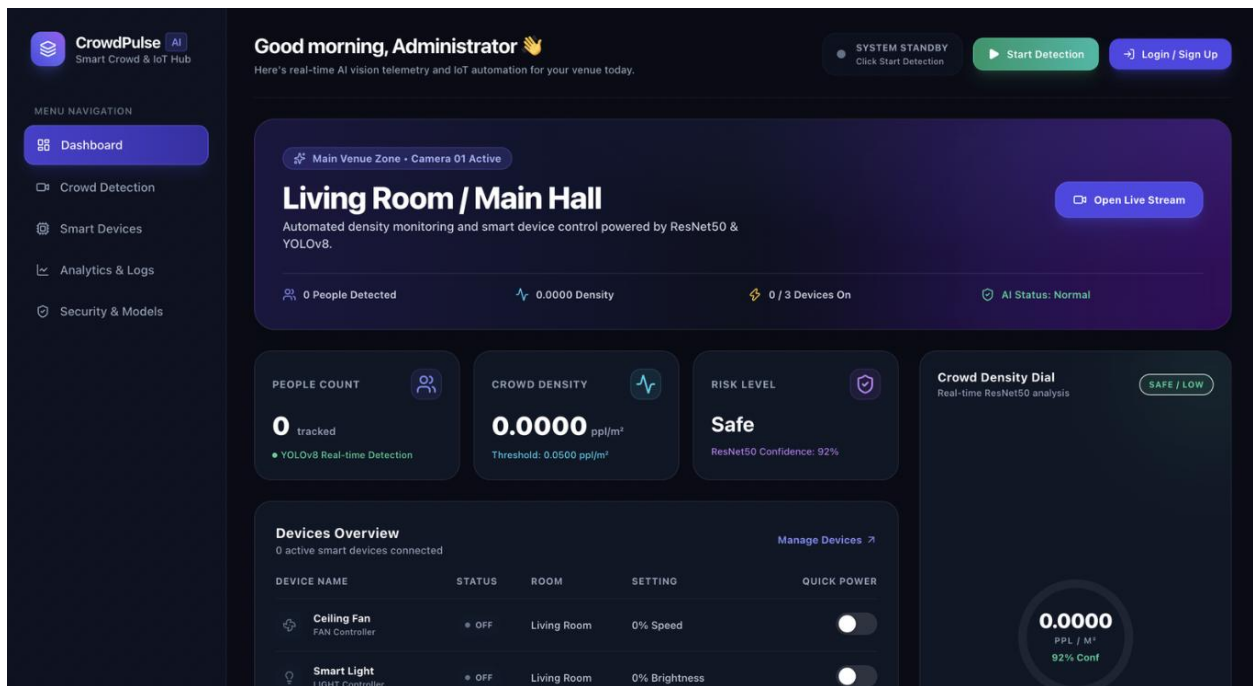


Figure 6.5 Dashboard Output

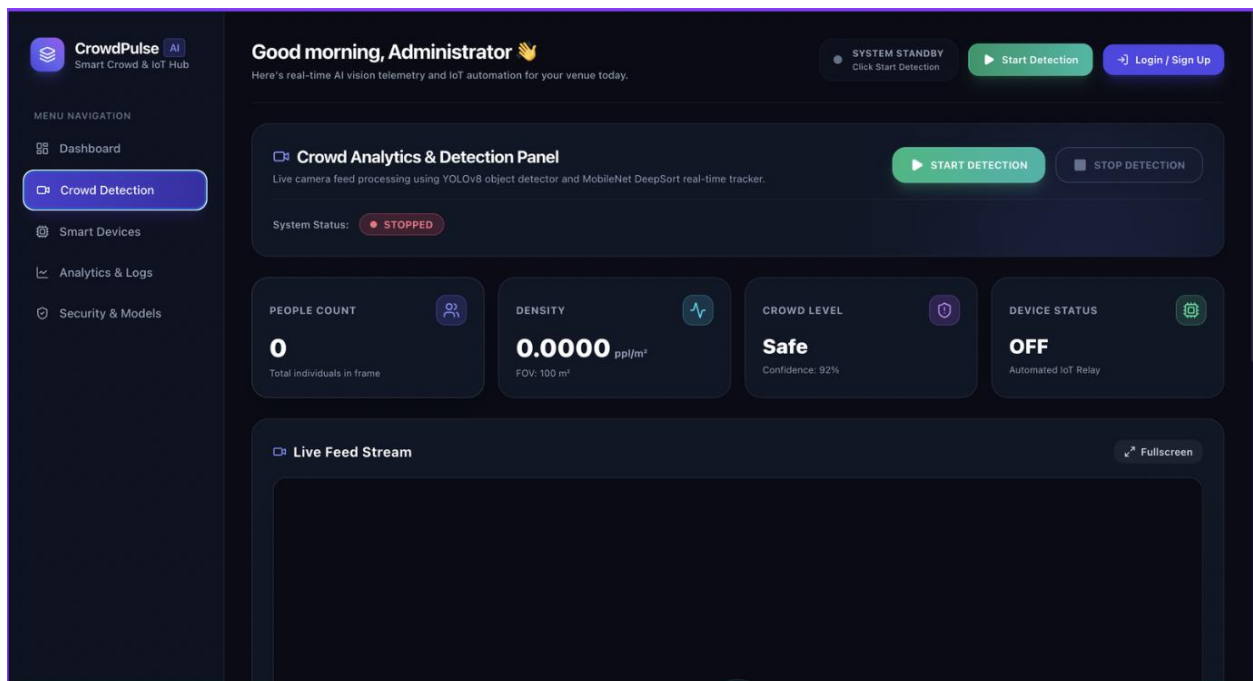


Figure 6.6 Crowd Detection Output

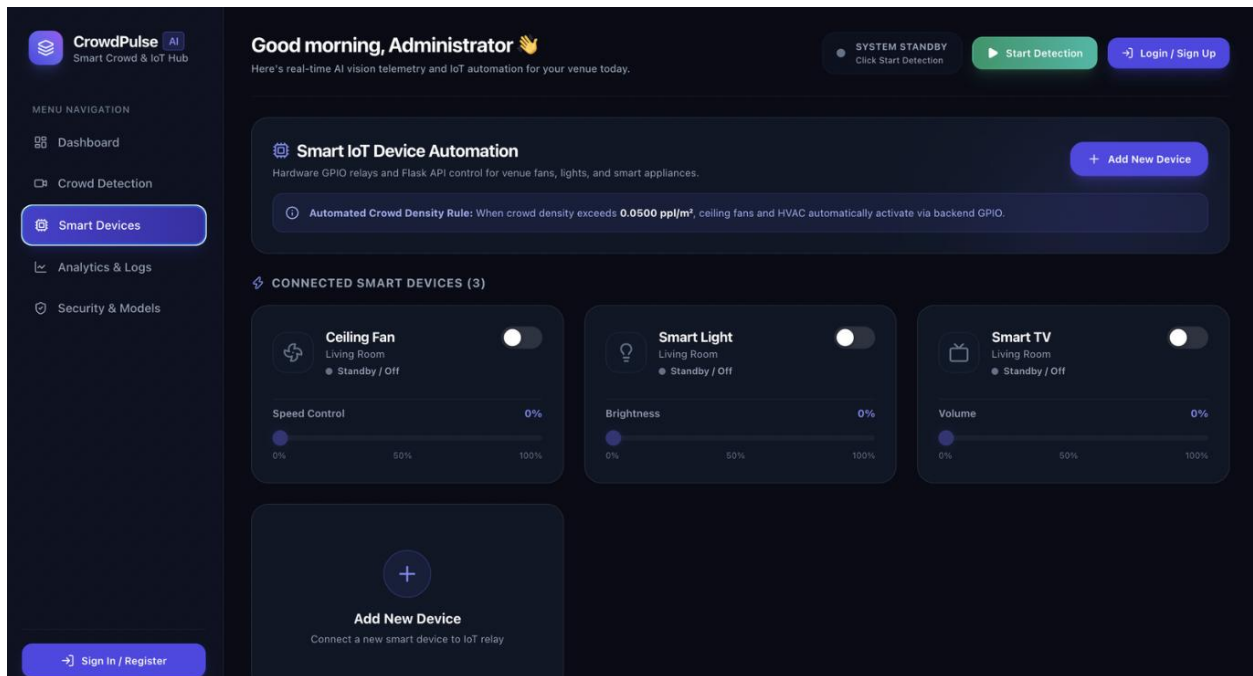


Figure 6.7 Home Controls Output

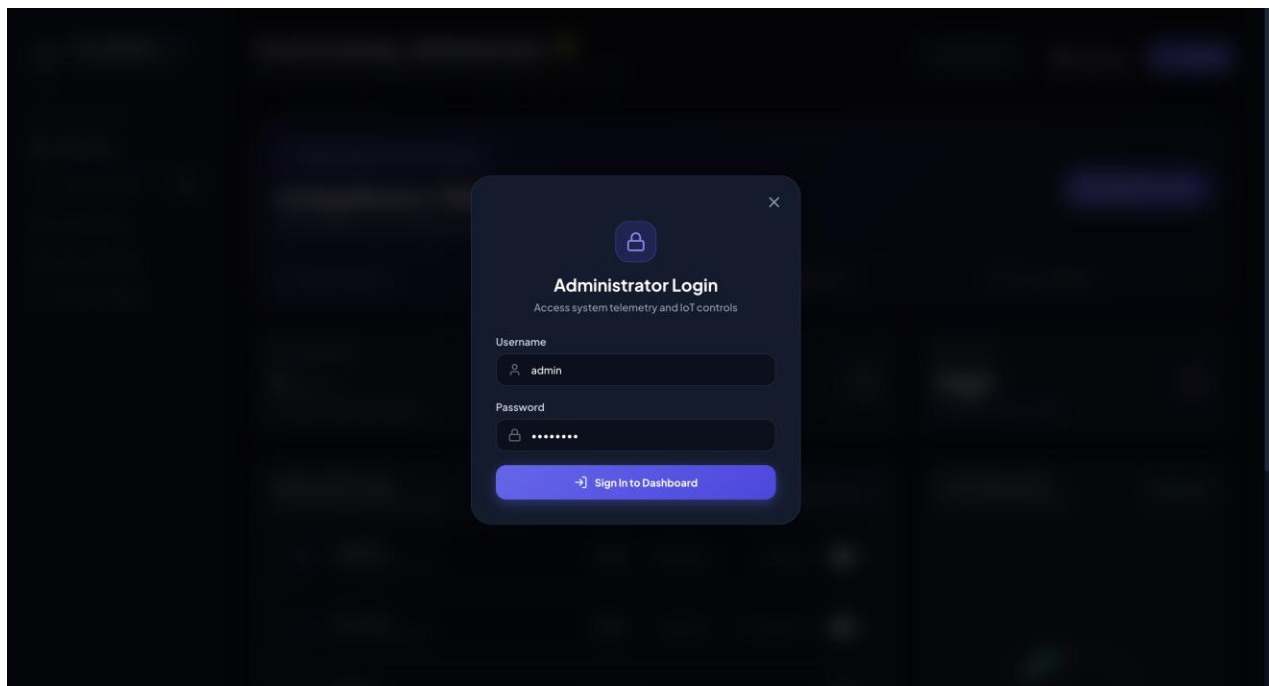


Figure 6.8 Admin Login Output

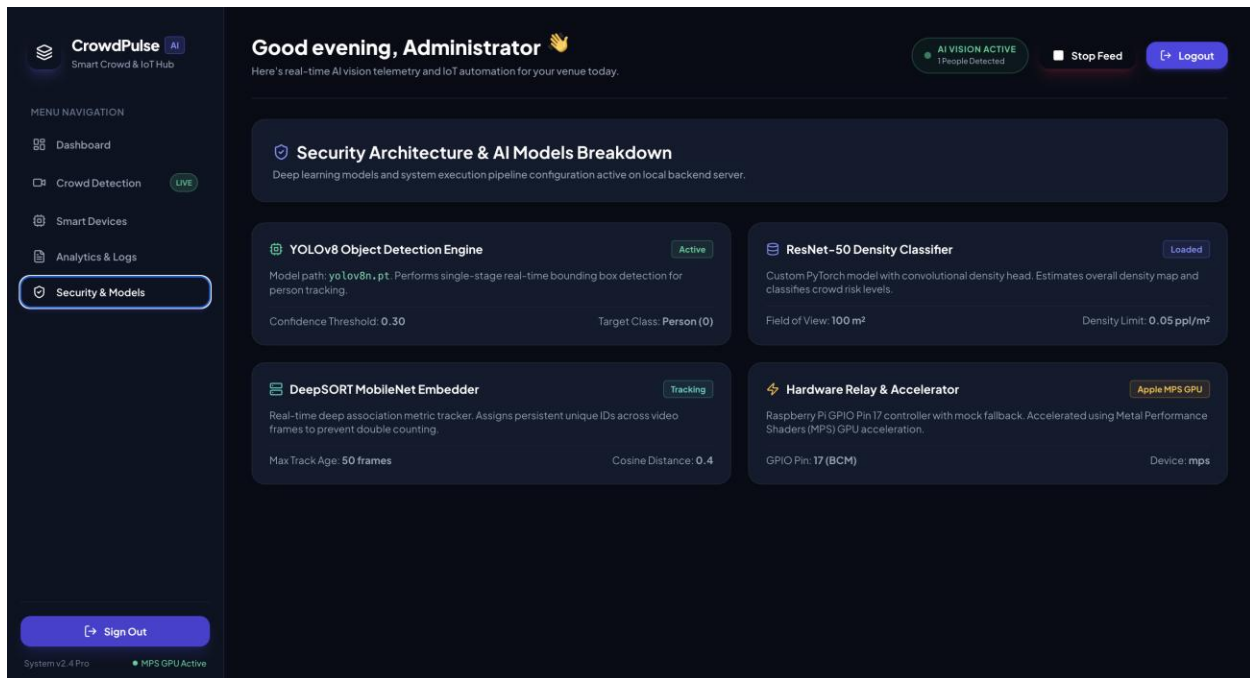


Figure 6.9 Working Models Output

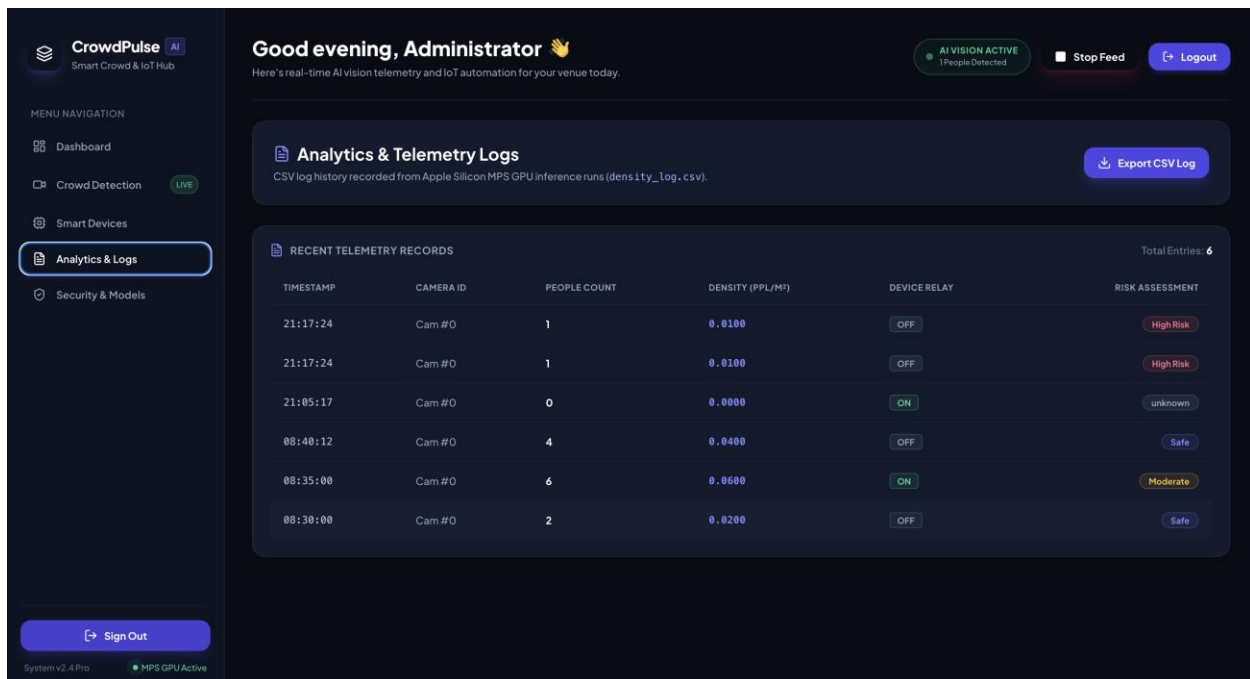


Figure 6.10 Analytics & logs Output